Lecture 5
Flow matching

Deep Neural Networks · University of Warsaw · 2026/27 · Mateusz Wyszyński

Motivation

Generative AI

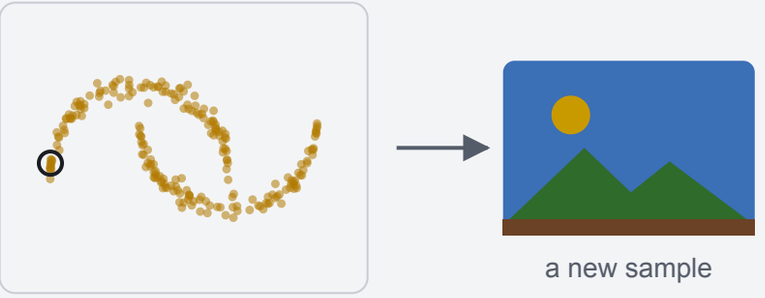
```
prompt: two black robot arms on a wooden
table folding a cardboard box, bright
university robotics lab, photorealistic
```



generated image step 50 / 50

Today's framework: generation = sampling from a distribution

$$z \sim q$$



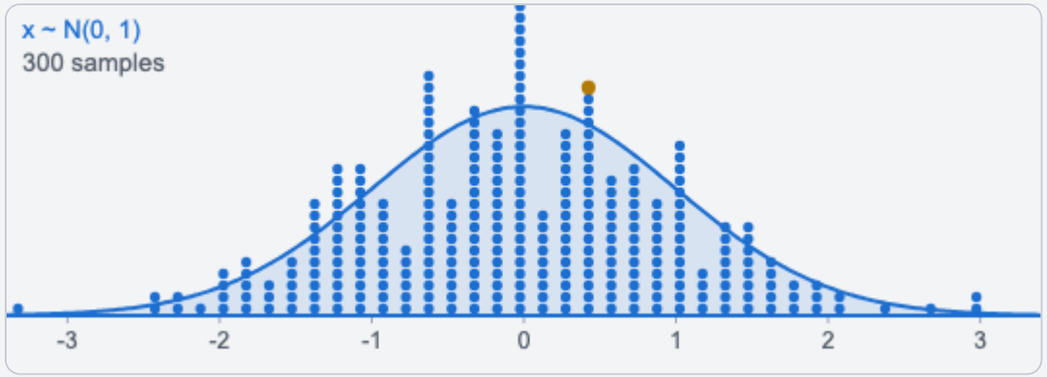
a new sample

$q : \mathbb{R}^d \to \mathbb{R}_{\geq 0}$ — we do not know this density, but we have samples of it

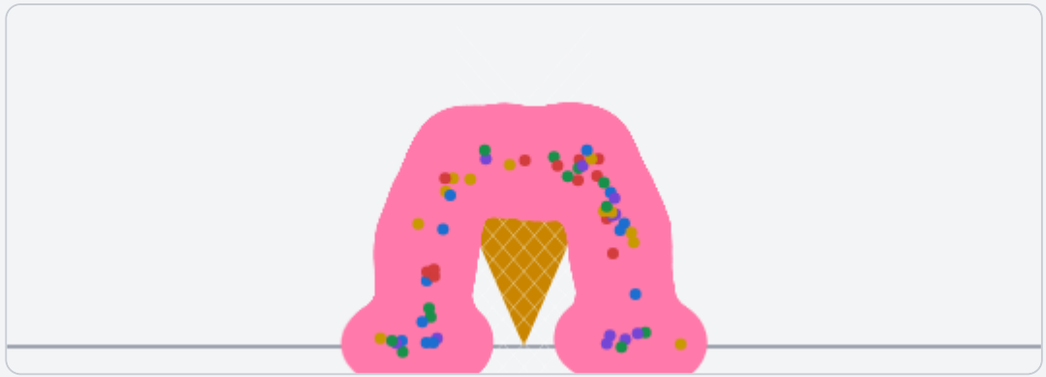
> Hence in the end we do not necessarily need to know the distribution. We need to be able to **sample** from it!

The goal and general idea

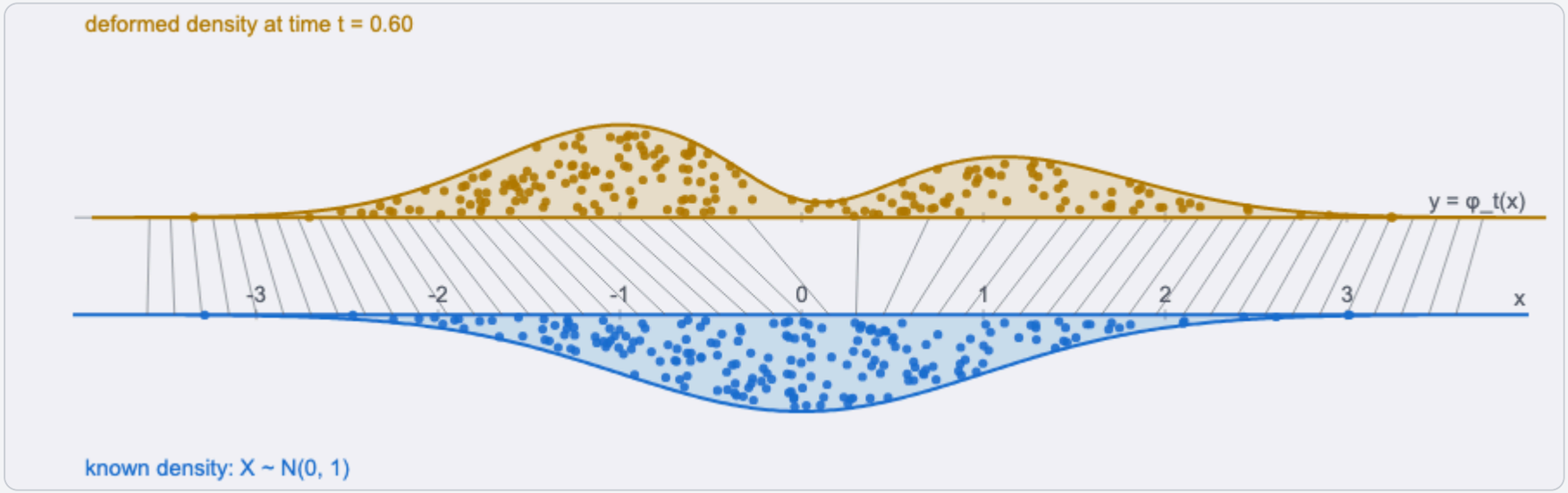
We can easily sample from known distributions



Deformable simulations: studied for decades



Combine the two ideas to generate samples from the distribution of interest



Overview

1 · preliminaries

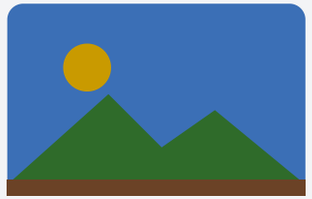
deformation, probability

2 · learning deformation

likelihood, residual flows, flow matching

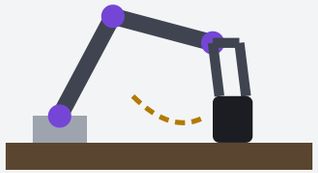
3 · extensions, applications & discussion

diffusion, conditioning, physical AI



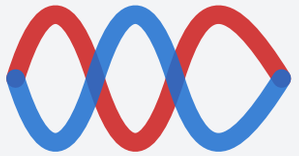
images & video

Stable Diffusion 3, Flux (flow matching); Sora, Veo (diffusion)



robot control

π0, π0.5 (flow matching)



proteins & molecules

RFdiffusion, AlphaFold 3 (diffusion)



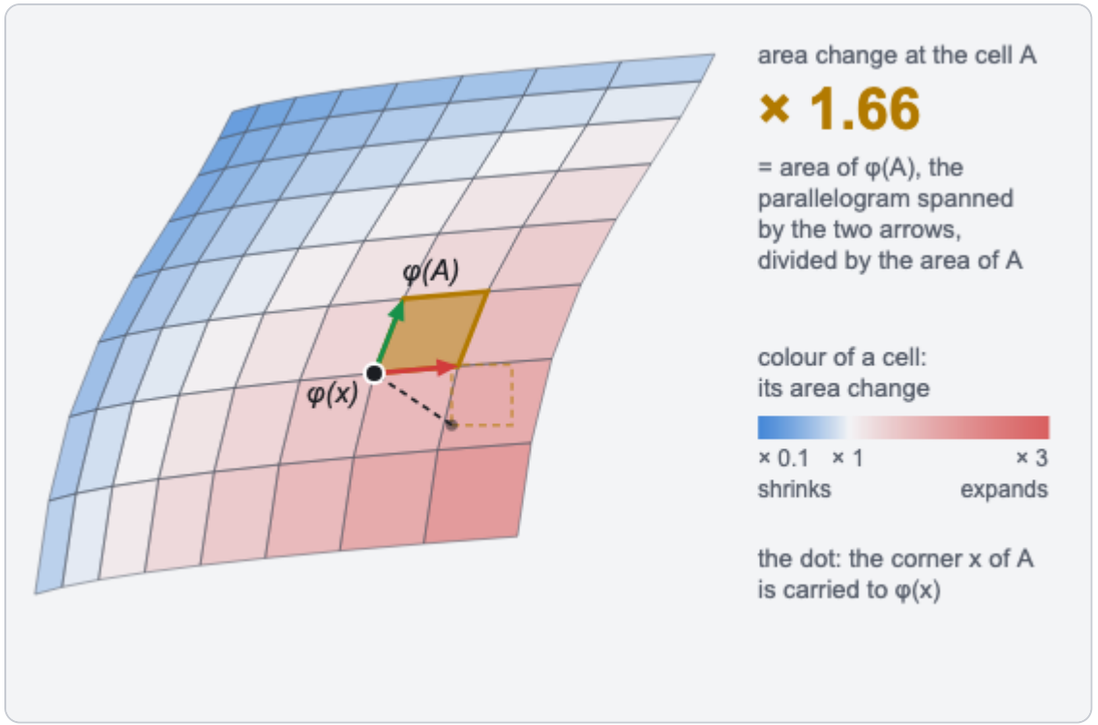
audio

Voicebox (flow matching), Stable Audio (diffusion)

Part 1

Preliminaries

Deforming probability



$\mathbb{P}[X \in A] = \mathbb{P}[Y \in \varphi(A)]$: the mass moves with the map. Arrows: the two sides of A after the map, i.e. the columns of the Jacobian $D\varphi$ times the side length (for small A). The area they span, relative to A, is the local area change $|\det D\varphi|$.

Consider an invertible map between random variables:

$$Y = \varphi(X)$$

Probabilities have to stay the same:

$$\mathbb{P}[Y \in B] = \mathbb{P}[X \in A] \implies \int_B p_Y(y)\,dy = \int_A p_X(x)\,dx$$

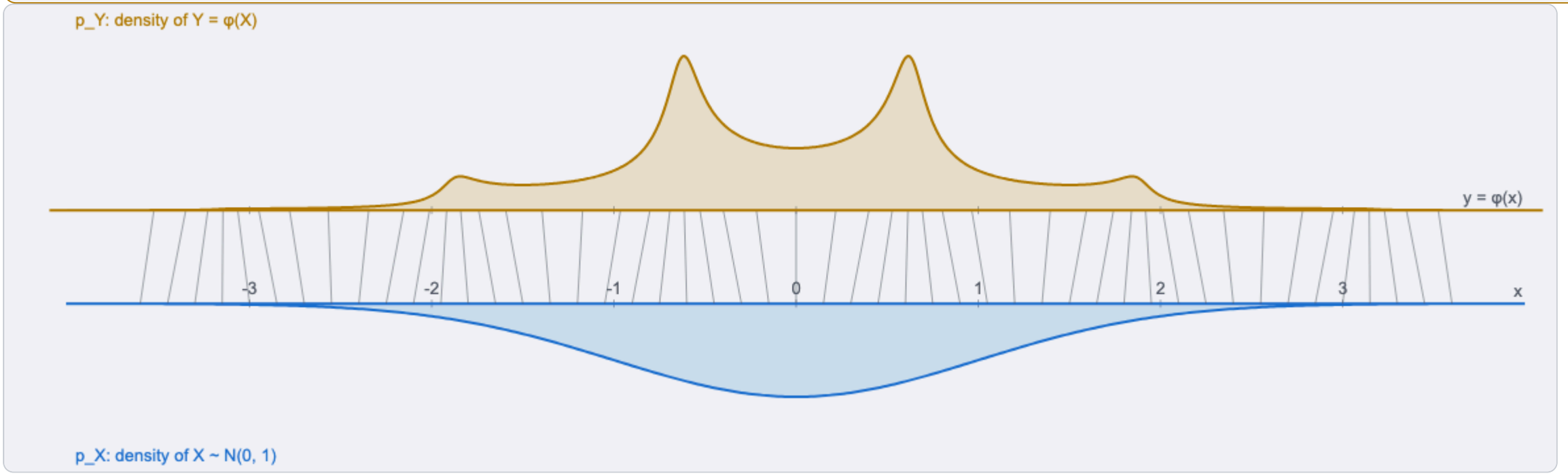
Substitute $x = \varphi^{-1}(y)$ on the right:

$$\int_A p_X(x)\,dx = \int_B p_X\left(\varphi^{-1}(y)\right) \left|\det D\varphi^{-1}(y)\right| dy$$

$$p_Y(y) = p_X\left(\varphi^{-1}(y)\right) \left|\det D\varphi^{-1}(y)\right|$$

Deforming probability: example

$$p_Y(y) = p_X\left(\varphi^{-1}(y)\right) \left|\det D\varphi^{-1}(y)\right|$$



$\phi(x) = x + s \cdot \sin(t \cdot x)$ with **s = 0.1**, **t = 5**

$\phi'(x) = 1 + s{\cdot}t{\cdot}\cos(t{\cdot}x) \in [0.5, 1.5]$ · invertible since $|s{\cdot}t| < 1$

> **Question** A unimodal density became multimodal. How was this possible? The mass near x lands near $y = \varphi(x)$, squeezed into an interval $|\varphi'(x)|$ times as long: $p_Y(y) = p_X(x) \,/\, |\varphi'(x)|$. Where φ compresses, density piles up.

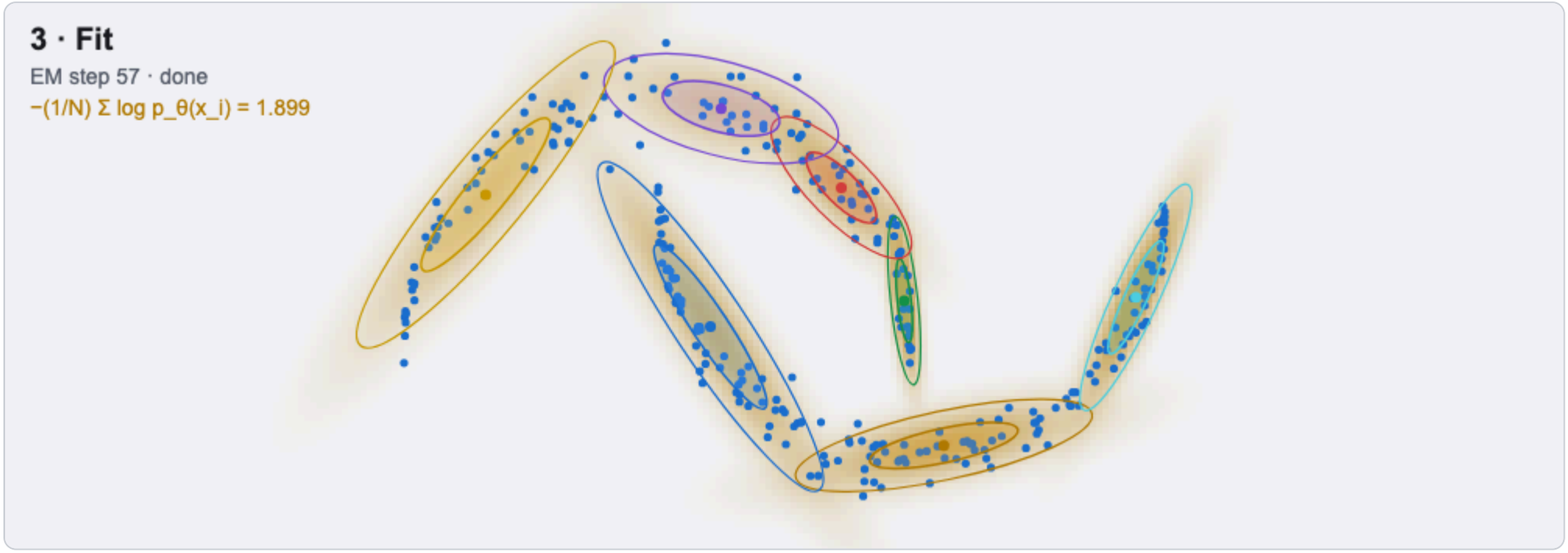
Part 2

Learning deformation

A simple idea how one could learn the deformed distribution

A **parametrised density**, e.g. a mixture of Gaussians:

$$p_\theta(x) = \sum_{k=1}^{K} \pi_k \, \mathcal{N}(x;\, \mu_k, \Sigma_k), \qquad \theta = \{\pi_k, \mu_k, \Sigma_k\}_{k=1}^{K}$$



Question 1. What does “fit the data” mean exactly?

Question 2. Is the mixture of Gaussians the best choice?

When is the fit good? KL divergence

When our model distribution p_θ is close to the unknown target distribution q:

$$D_{\mathrm{KL}}(q \,\|\, p_\theta) = \int q(x)\, \log \frac{q(x)}{p_\theta(x)}\, dx = \mathbb{E}_{x\sim q}\Big[\log \frac{q(x)}{p_\theta(x)}\Big]$$

the Kullback–Leibler divergence

Question Why does this measure make sense?

Consider a coin flip:

H H T H T T H … N_H heads, N_T tails

Compare the probabilities under the true distribution q against the model p:

$$\Big(\prod_{i=1}^{N} \frac{q(x_i)}{p(x_i)}\Big)^{1/N} = \Big(\frac{{q_H}^{N_H}\, {q_T}^{N_T}}{{p_H}^{N_H}\, {p_T}^{N_T}}\Big)^{1/N}$$

Take the log and consider $N \to \infty$ (so, e.g., $N_H/N \to q_H$):

$$\frac{N_H}{N} \log \frac{q_H}{p_H} + \frac{N_T}{N} \log \frac{q_T}{p_T} \longrightarrow \sum_x q(x) \log \frac{q(x)}{p(x)}$$

KL divergence in practice

$$D_{\mathrm{KL}}(q \,\|\, p_\theta) = \mathbb{E}_{x\sim q}\left[\log \frac{q(x)}{p_\theta(x)}\right]$$

Question Do we have to minimize KL divergence directly?

Split the log:

$$D_{\mathrm{KL}}(q \,\|\, p_\theta) = \underbrace{\mathbb{E}_{x\sim q}[\log q(x)]}_{\text{does not depend on } \theta} \underbrace{-\mathbb{E}_{x\sim q}[\log p_\theta(x)]}_{\text{cross-entropy}}$$

The first term is a property of the data alone (its entropy, up to sign): unknown, but the same for every θ. Only the second term moves when we change the model.

Minimizing D_{KL} = minimizing the cross-entropy. **Loss:**

$$\mathcal{L}(\theta) = -\,\mathbb{E}_{x\sim q}[\log p_\theta(x)]$$

Question An expectation over the unknown q: how do we compute it during training?

Monte Carlo:

$$\mathbb{E}_{x\sim q}[f(x)] \approx \frac{1}{N}\sum_{i=1}^{N} f(x_i), \qquad x_i \sim q$$

samples drawn according to the distribution

Question How do we get samples from q?

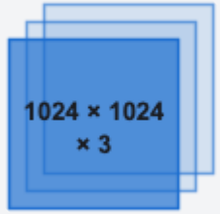
Data! The training set is a sample from q:

$$\mathcal{L}(\theta) \approx -\frac{1}{N}\sum_{i=1}^{N} \log p_\theta(x_i)$$

the cross-entropy of Lab 4, now for a density.

Is the mixture of Gaussians the best choice?

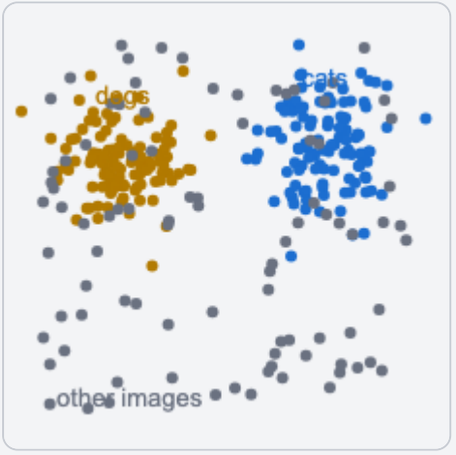
Parameter cost. A Gaussian in d dimensions needs at least d numbers, for its mean alone. **How many Gaussians** does the budget of a 7B network buy?

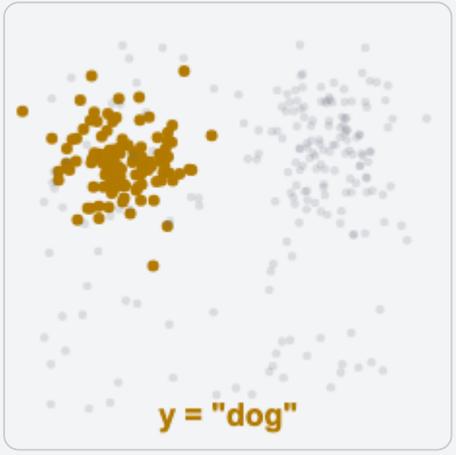


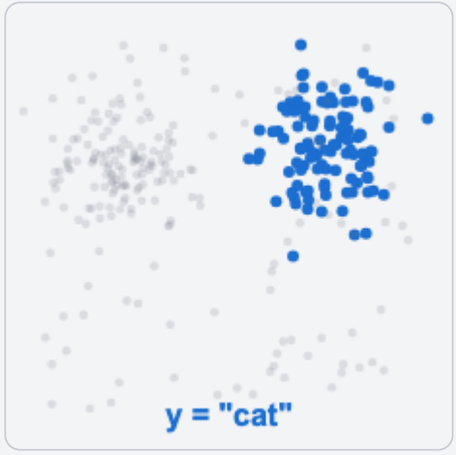
photo, 1024 × 1024 × 3

$$\frac{\textbf{7 000 000 000}\ \text{parameters}}{\textbf{3 145 728}\ \text{numbers per Gaussian}\ (d \text{ with } d = 3\,145\,728)} = \textbf{2 225}\ \text{Gaussians}$$

training set of 1 billion images → one Gaussian per 449 390 images





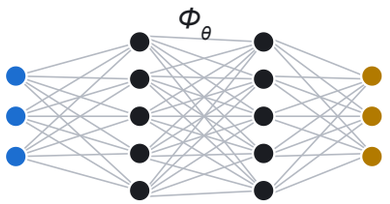


We might want to learn a **family of distributions** $\{ p_\theta(\cdot \mid y) \}_y$ (e.g., one distribution per prompt). A mixture has no natural extra input path to allow conditioning.

Residual flows

We can try with a neural network instead:

$$X \sim p_{\text{init}}, \qquad \Phi_\theta(X) \sim p_\theta$$



Question Will any network Φ_θ work?

It has to be a valid probability deformation:

$$p_\theta(x) = p_{\text{init}}\left(\Phi_\theta^{-1}(x)\right) \left| \det D\Phi_\theta^{-1}(x) \right|$$

- it has to be **invertible**…
- …and we must be able to compute the **determinant** efficiently

One way to do it: residual networks (Lecture 4)



$$\varphi = \varphi_N \circ \cdots \circ \varphi_1, \qquad \varphi_k(x) = x + \delta\, u_k(x)$$

If $u_{k,i}$ depends only on $x_1, \ldots, x_i$, we can invert one coordinate at a time: φ_k is invertible as soon as no diagonal entry vanishes, and the triangular Jacobian makes the determinant a product:

$$D\varphi_k = \begin{pmatrix} 1+\delta\,\partial_1 u_{k,1} & 0 & \cdots & 0 \\ * & 1+\delta\,\partial_2 u_{k,2} & & \vdots \\ \vdots & & \ddots & 0 \\ * & \cdots & * & 1+\delta\,\partial_d u_{k,d} \end{pmatrix}$$

$$\det D\varphi_k = \prod_{i=1}^{d} \left(1 + \delta\,\partial_i u_{k,i}\right)$$

$$\mathcal{L}(\theta) = -\,\mathbb{E}_{x\sim q}\left[\log p_{\text{init}}\left(\varphi^{-1}(x)\right) + \sum_{k=1}^{N} \log \left| \det D\varphi_k^{-1} \right| \right]$$

A residual step is an Euler step

residual block (Lecture 4)

$$x_{k+1} = x_k + \delta\, u_k(x_k)$$

Euler's method for $\frac{dx}{dt} = u(t, x)$

$$x(t+h) \approx x(t) + h\, u\big(t, x(t)\big)$$

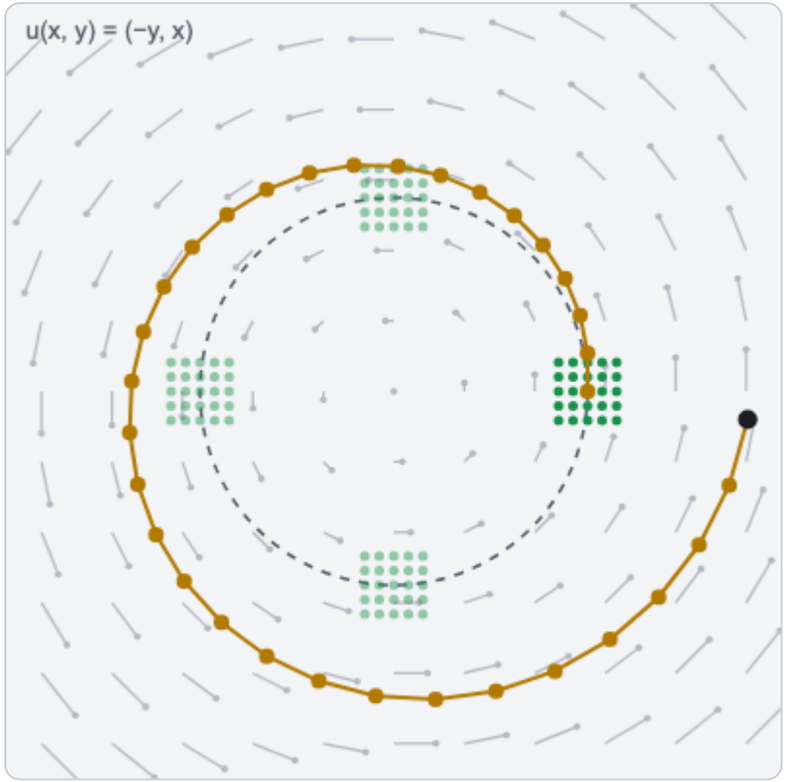
a numerical solver for an ordinary differential equation (ODE); the step h is the solver's choice

> The same formula: let $\delta \to 0$, $N \to \infty$ and a chain of residual blocks becomes a **continuous-time flow** driven by one velocity field u

Numerical integration: the step size matters

$$x_{k+1} = x_k + h\,u(x_k), \qquad h = \frac{T}{n}$$

Euler's method: n steps of size h over the time T. Here $u(x, y) = (-y,\, x)$ is a rotation and $T = 2\pi$ is one full turn, so the exact solution stays on its circle.



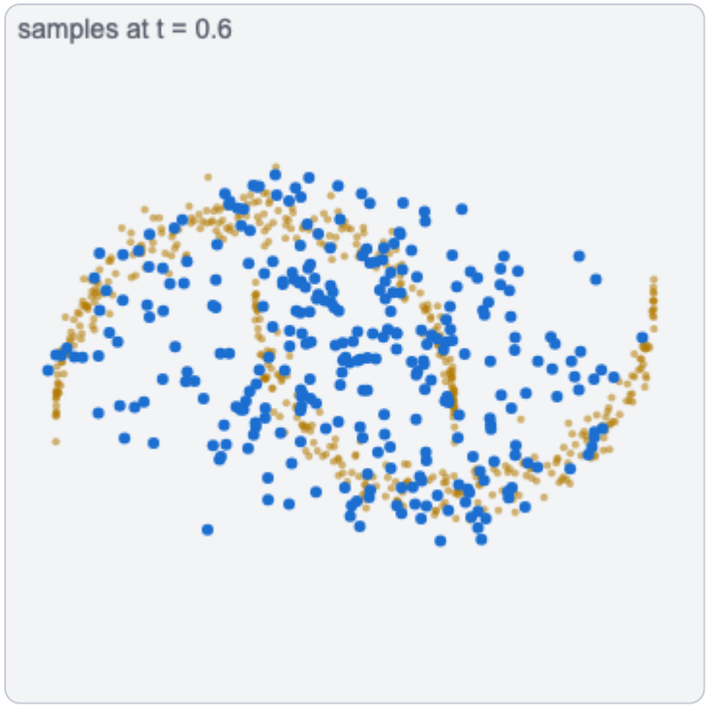
n = 32 Euler steps over one full turn:

h = 2π/32 = 0.2

- a patch carried by the exact flow (every point circles at the same speed) · dashed: the exact orbit
- Euler path · distance from the origin: 1.6 → **2.93** after one turn (exact: 1.6)

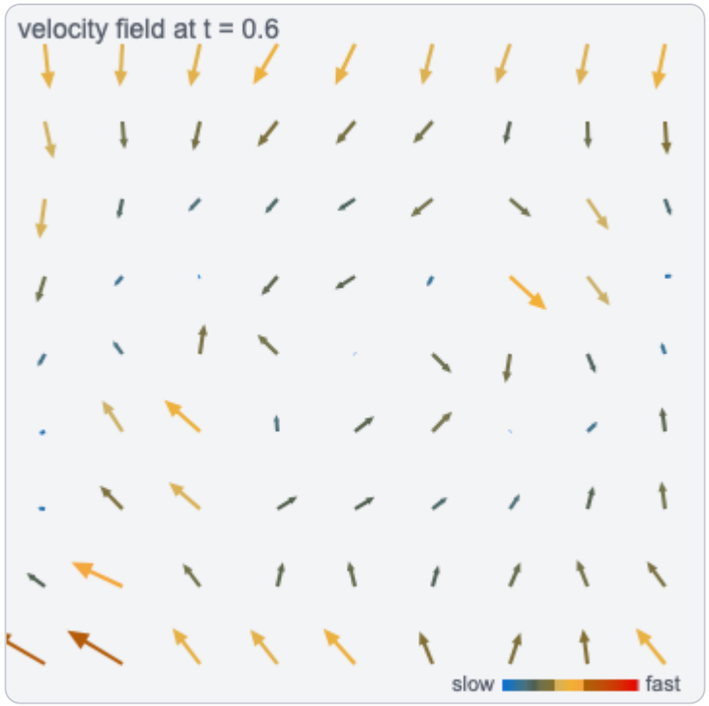
Euler follows the tangent, so large steps drift away. More steps = a more faithful flow, at the cost of more evaluations of u. Remember this for sampling.

Probability flows



Flow

$$\underbrace{\varphi}_{\text{flow}} : \underbrace{[0,1]}_{\text{time}} \times \underbrace{\mathbb{R}^d}_{\text{points in space}} \to \mathbb{R}^d$$

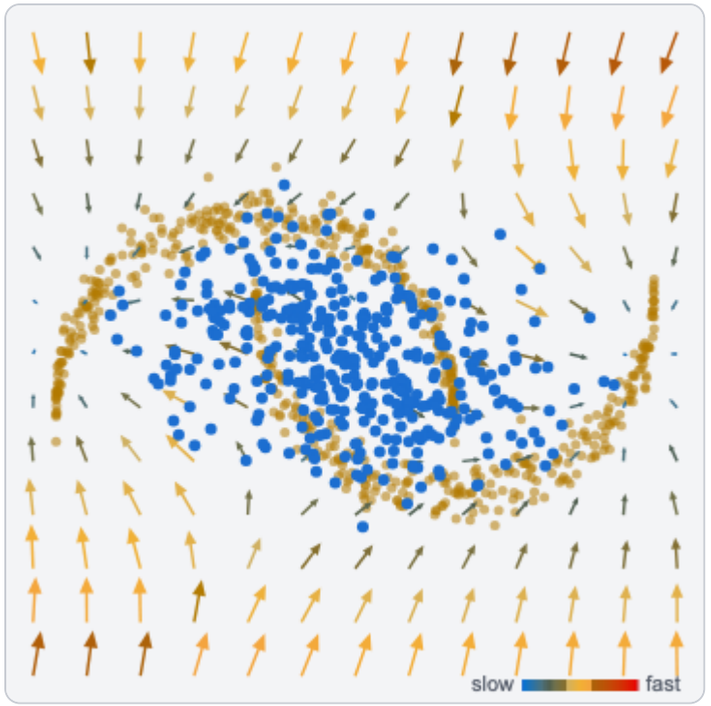


Underlying time-dependent vector field

$$\frac{d\varphi_t(x_t)}{dt} = u_t(x_t)$$

> A shift in the approach: learn the vector field and use numerical integration to determine the probability

A flow: a velocity field moving every point



Gaussian → two moons

time **t = 0.5**

● the same samples, moved along the learned field · ● data · arrows: vθ(t, x)

$$\frac{d}{dt}\varphi_t(x) = u_t\big(\varphi_t(x)\big), \qquad \varphi_0(x) = x$$

Glossary:

- $\varphi_t(x)$: where the point that started at x is at time t
- $u_t(x) = u(t, x)$: the **velocity field** — the thing we directly learn

New: the **probability path** p_t = the distribution of $\varphi_t(X_0)$ at time t. $p_0 = \mathcal{N}(0, I)$, and we want $p_1 \approx q$, i.e., the target distribution.

What we get

any network

$v_\theta(t, x)$ needs no invertibility: an ODE can always be run backwards in time

an arbitraty number of steps

how many numerical integration steps, how big, how computed is not baked in into the network

the density is derived implicitly

$$\log p_1(x_1) = \log p_0(x_0) - \int_0^1 \nabla \cdot u_t(x_t)\, dt,$$

the continuous version of

$$\log p_{\text{init}}\left(\varphi^{-1}(x)\right) + \sum_{k=1}^{N} \log \left| \det D\varphi_k^{-1} \right|$$

Continuous normalizing flows train v_θ by maximum likelihood with the new formula: every training step integrates the ODE and the divergence of a network, for every data point. Correct, but slow.

Flow matching (2022): train the same v_θ without ever computing a density.

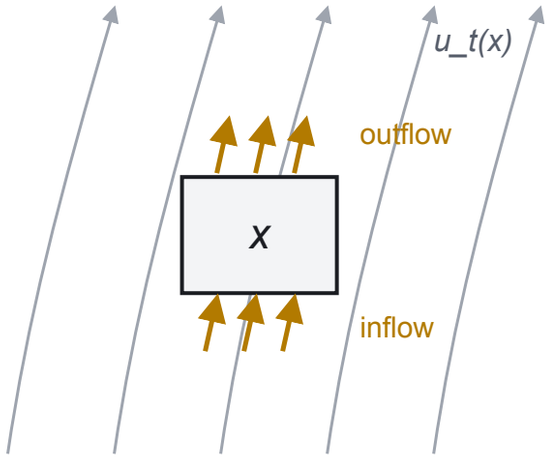
$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t\sim U[0,1],\ x\sim p_t} \left\| v_\theta(t, x) - u_t(x) \right\|^2$$

Why a velocity field defines a probability path OPTIONAL

$$\frac{\partial p_t(x)}{\partial t} = -\nabla \cdot \left(p_t(x)\, u_t(x)\right)$$

The **continuity equation**: the density at x changes exactly by the net inflow of probability mass carried by the velocity field. Mass is neither created nor destroyed — only moved.

u_t generates p_t when this holds. The equation is linear in $(p_t,\ p_t u_t)$ — we use that on the marginal-velocity slide.



Flow matching

Suppose we knew a velocity field u_t that carries $\mathcal{N}(0, I)$ onto p_{data} along a probability path p_t. Fit a network to it by least squares:

$$\mathcal{L}_{\text{FM}}(\theta) = \mathbb{E}_{t\sim U[0,1],\ x\sim p_t} \left\| v_\theta(t, x) - u_t(x) \right\|^2$$

$t \sim U[0, 1]$
a random moment

$x \sim p_t$
a point where the flow actually passes at time t

$v_\theta(t, x)$
the network: any architecture, inputs t and x

$u_t(x)$
the target velocity

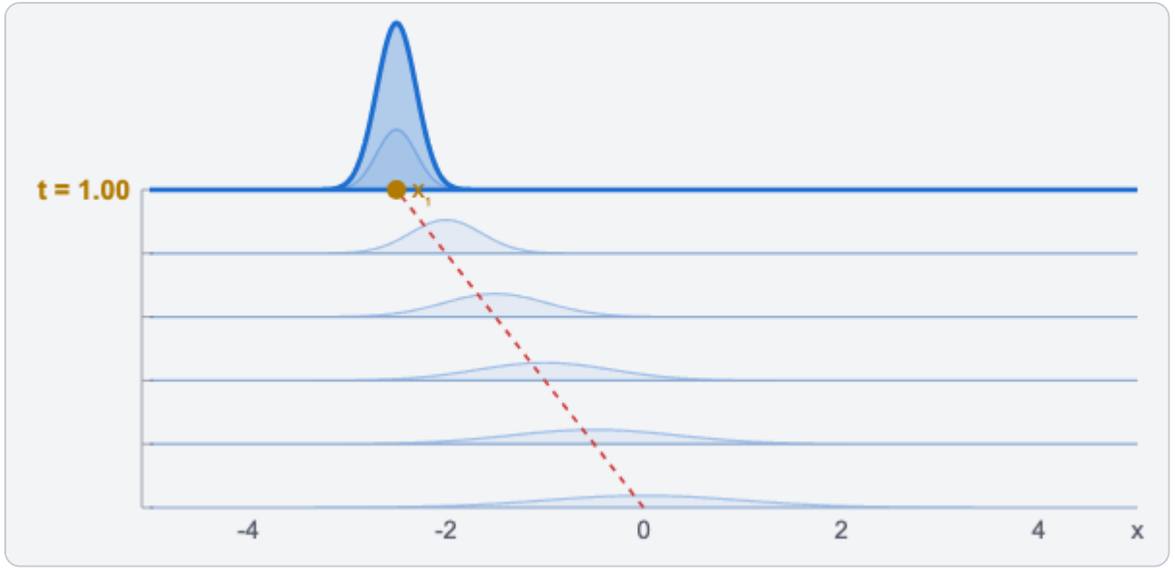
Problems:

- We know neither p_t (no way to sample) nor u_t (no target).
- Infinitely many flows go from noise to data, and
- we cannot write down a single one of them...

Question What can we do?

Single sample simple paths

Can design reasonable paths if we consider datasets with single datapoints:



$p_t(x \mid x_1) = \mathcal{N}(\mu_t(x_1), \sigma_t^2)$ with $x_1 = -2.5$

linear (optimal transport) path: $\mu_t(x_1) = t \cdot x_1$, $\sigma_t = 1 - (1 - \sigma_{min})\, t$

conditional probability path $p_t(x \mid x_1)$: from $\mathcal{N}(0, I)$ at $t = 0$ to a distribution gathered around x_1 at $t = 1$ (our design choice)

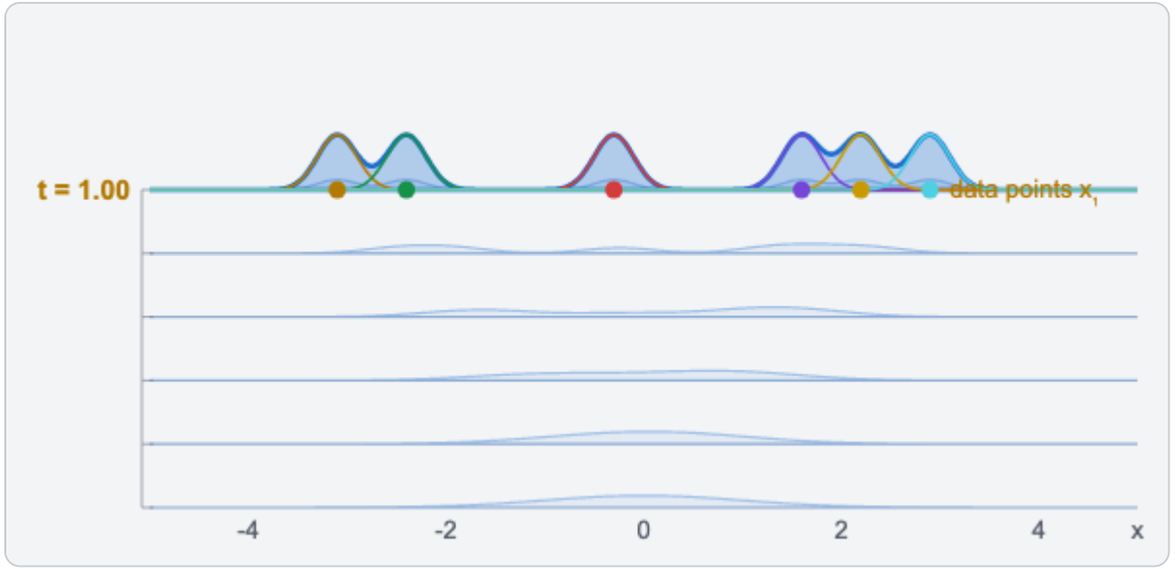
As before, a related **conditional vector field** $u_t(x \mid x_1)$ determines the conditional deformation (continuity equation):

$$\frac{\partial}{\partial t} p_t(x \mid x_1) = -\nabla \cdot \Big(p_t(x \mid x_1)\, u_t(x \mid x_1) \Big)$$

For some families of conditional paths exact formulas for $u_t(x \mid x_1)$ can be derived (see animation above).

Mixtures of single datapoint paths

Average the simple paths over all data points:



pt(x) = average of pt(x | x_1) over the **6** data points

linear (optimal transport) path: $\mu t(x_1) = t \cdot x_1$, $\sigma t = 1 - (1 - \sigma\min)\, t$

$$p_t(x) = \int p_t(x \mid x_1)\, p_{\text{data}}(x_1)\, dx_1$$

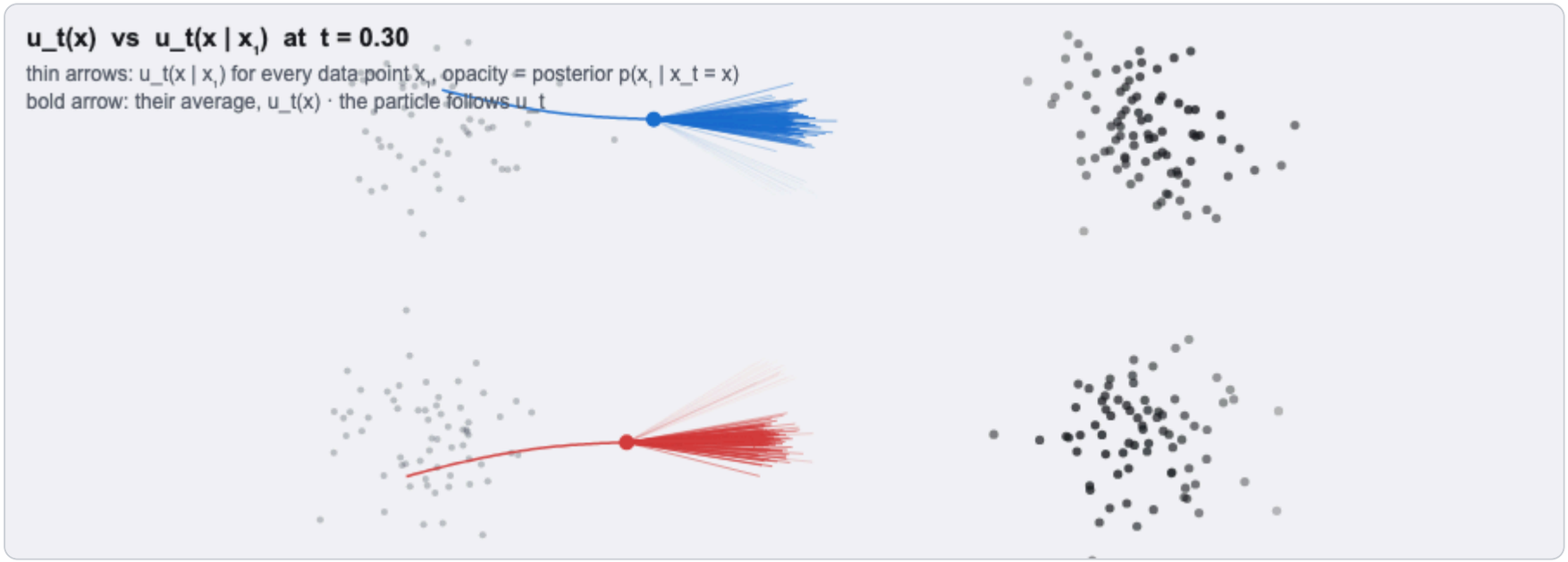
- $t = 0$: every conditional path starts at $\mathcal{N}(0, I)$, so $p_0 = \mathcal{N}(0, I)$
- $t = 1$: a tiny Gaussian around each data point, so $p_1 \approx p_{\text{data}}$
- in between: a valid probability path from noise to data

> **Question** Which conditional distributions at $t = 1$ give exactly p_{data}? Dirac deltas, i.e. $\sigma_{\min} = 0$.

The velocity of the mixture

One can* show that at a given point on the trajectory the mixture velocity is the average of the simple velocities:

$$u_t(x_t) = \mathbb{E}_{p_{1|t}}\left[\, u_t(x_t \mid x_1) \,\right] = \int u_t(x_t \mid x_1)\, \frac{p_t(x_t \mid x_1)\, q(x_1)}{p_t(x_t)}\, dx_1 = \int u_t(x_t \mid x_1)\, p_t(x_1 \mid x_t)\, dx_1$$



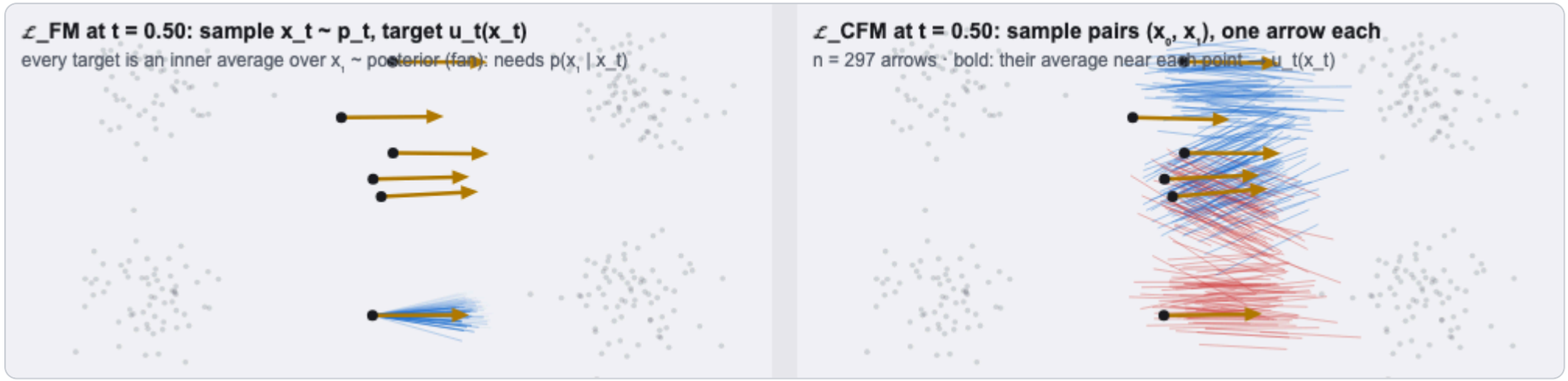
Question Can we sample from $p_t(x_1 \mid x_t)$? Why yes/no?

* relatively easily; again, via the continuity equation

The trick: regress on the conditional velocity instead

Surprisingly, one can* show that:

$$\underbrace{\mathbb{E}_{t,\, x\sim p_t}\left\|v_\theta(t,x)-u_t(x)\right\|^2}_{\mathcal{L}_{\mathrm{FM}}(\theta)\ \text{(unconditional loss)}} = \underbrace{\mathbb{E}_{t,\, x_1\sim p_{\mathrm{data}},\, x\sim p_t(\cdot|x_1)}\left\|v_\theta(t,x)-u_t(x\mid x_1)\right\|^2}_{\mathcal{L}_{\mathrm{CFM}}(\theta)\ \text{(conditional loss)}} + \mathrm{const}$$



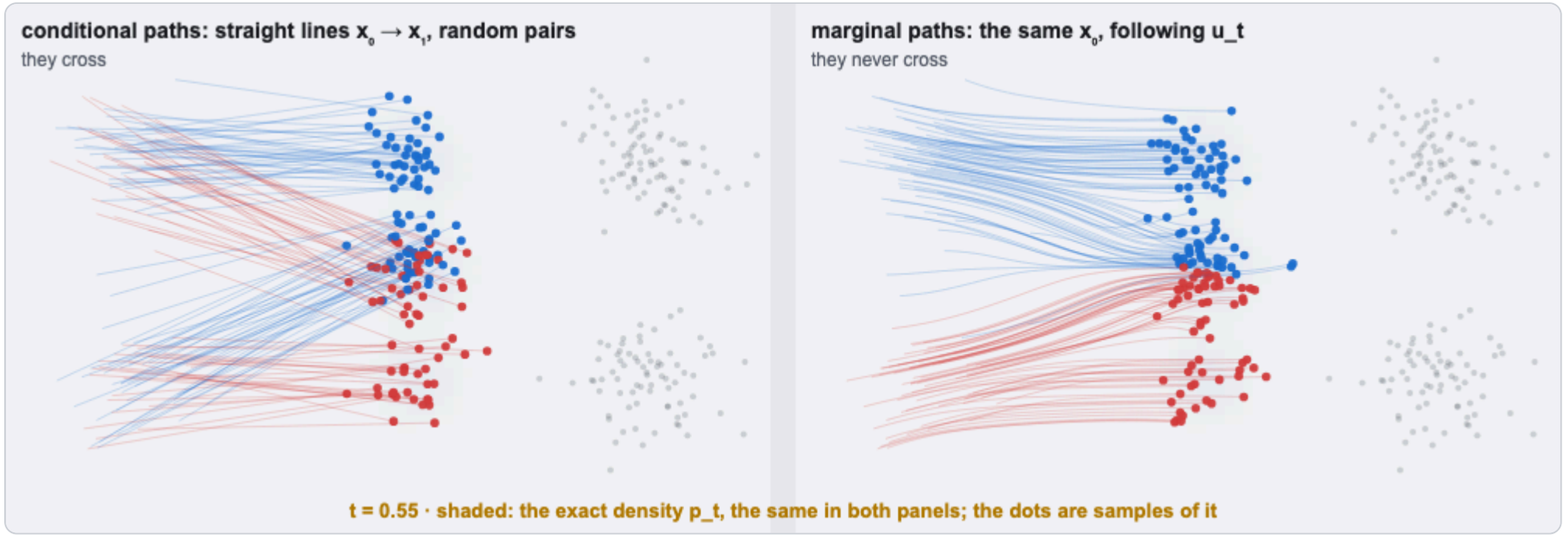
Think of changing the sampling scheme: instead of sampling $x \sim p_t$, estimating $\bar{u}_t(x)$ for each x and then estimating the loss based on the estimated targets, we sample single **conditional arrows** $u_t(x \mid x_1)$ all over the space, compute the squared error for each and compute the average to estimate the loss.

> With reasonable conditional paths and enough data we can estimate the conditional loss and compute gradients without ever estimating the true vector field.

*again, relatively easily

The same p_t, different paths

The conditional arrows (left) move points along straight lines that can cross. The marginal field moves the same points along curves that never cross. At every t, both in the right and in the left animation, we have a sample from the same p_t. The resulting samples are not the same, but the underlying distributions are



The whole algorithm

training

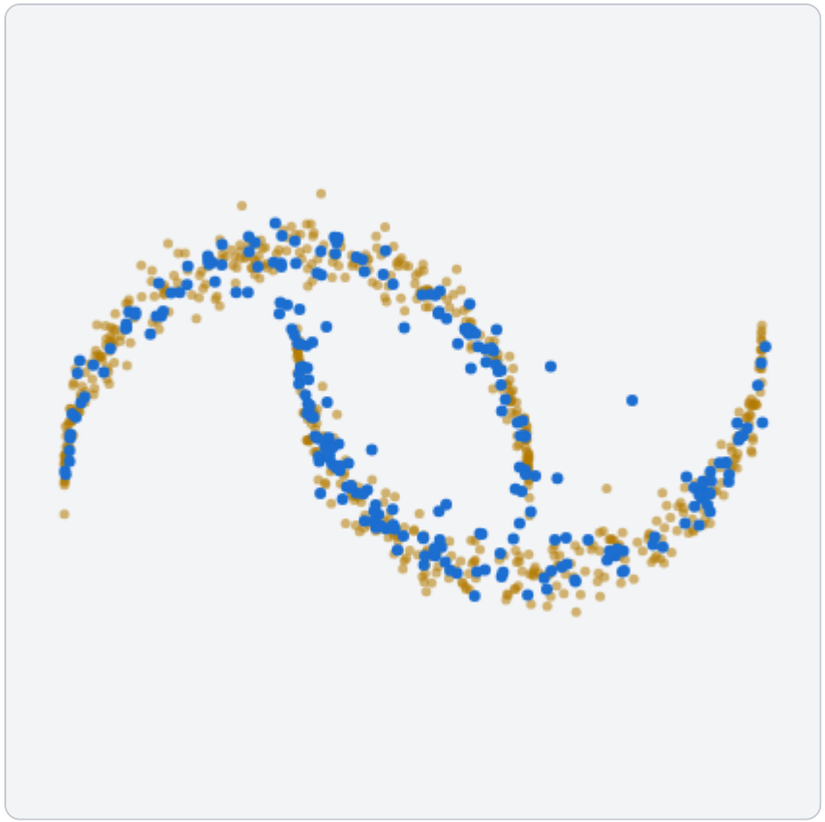
```
initialize θ randomly
repeat:
    x1 ← a minibatch of data
    x0 ← noise  N(0, I)          # same shape as x1
    t  ← Uniform[0, 1]            # one per example
    xt ← (1 − t) · x0 + t · x1
    L  ← ‖ v_θ(t, xt) − (x1 − x0) ‖²
    θ  ← θ − η · ∇_θ L
```

inference

```
x ← noise  N(0, I)
for k = 0, …, n−1:
    x ← x + (1/n) · v_θ(k/n, x)
return x
```

> During training: no likelihood, no Jacobian, no ODE. One network evaluation per example, like any regression. The ODE solver appears only during inference.

Watch it learn



Gaussian → two moons

MLP 3 → 64 → 64 → 64 → 2, **8706** parameters

pretrained (6,000 steps)

● samples generated by the current network (12 Euler steps) · ● data

A 3-layer MLP $v_\theta(t, x)$ trained in your browser with Adam on the conditional loss (backprop written by hand, as in Lecture 1). Blue: the samples it generates right now, with 12 Euler steps.

Inference: number of steps changes the result



64 integration steps

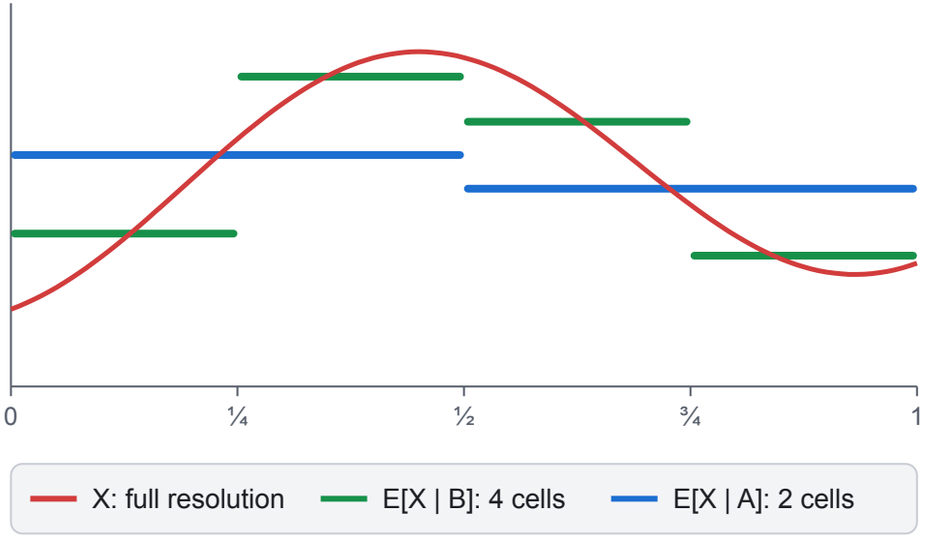
more steps follow the curves (dashed: the exact flow)

For the proofs: conditional expectation OPTIONAL

$\mathbb{E}[X \mid \mathcal{B}]$ is a **random variable**: the average of X over each cell you can resolve with the information $\mathcal{B}$. A camera metaphor: finer information, sharper picture.

$$\mathbb{E}\big[\,\mathbb{E}[X \mid \mathcal{B}] \,\big|\, \mathcal{A}\big] = \mathbb{E}[X \mid \mathcal{A}] \quad \text{whenever } \mathcal{A} \subseteq \mathcal{B}$$

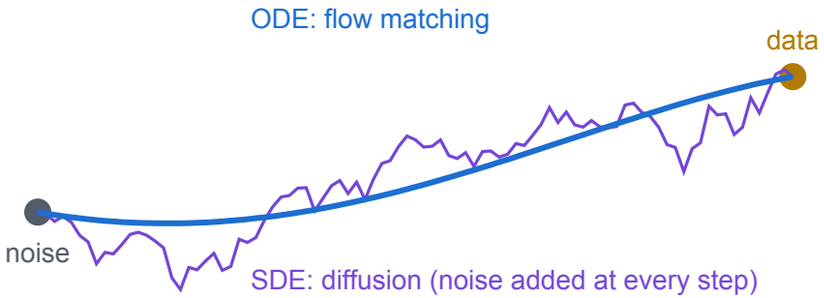
A low-resolution picture of a high-resolution picture is just the low-resolution picture; with no information at all, $\mathbb{E}\big[\mathbb{E}[X \mid \mathcal{B}]\big] = \mathbb{E}[X]$. The theorem on the previous slides is this identity applied to $\mathbb{E}[x_1 - x_0 \mid x_t]$.



Part 3

Extensions, Applications & Discussion

Diffusion models



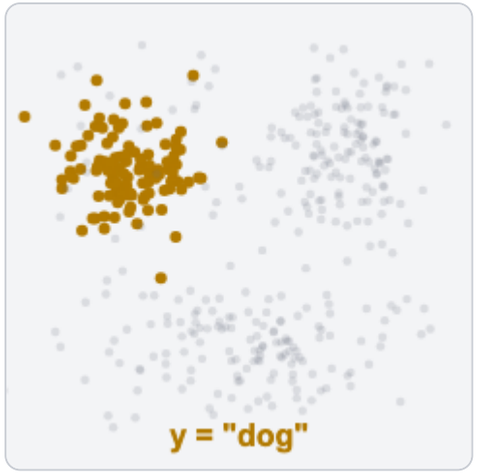
	flow matching	diffusion
sampling	ODE: deterministic steps	SDE: every step adds noise
what is learned	the velocity u_t	the score $\nabla_x \log p_t$ (or the noise)
training	the same trick: regress on a per-data-point target	

Conditional generation

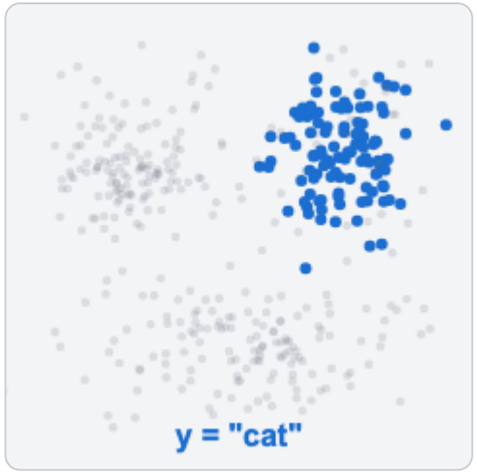
p_{data}: all images



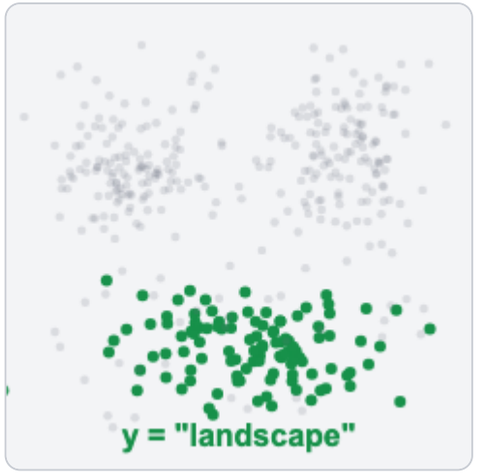
$p_{\text{data}}(\cdot \mid y = \text{“dog”})$



$p_{\text{data}}(\cdot \mid y = \text{“cat”})$



$p_{\text{data}}(\cdot \mid y = \text{“landscape”})$

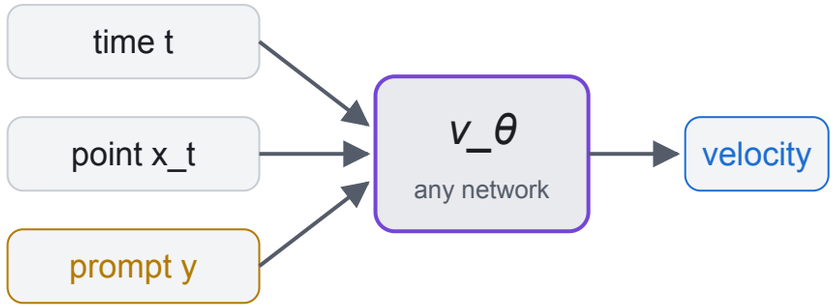


A prompt y picks one member of a **family of distributions** $\{ p_{\text{data}}(\cdot \mid y) \}_y$ indexed by y. We learn the whole family at once.

In general y = **conditioning tokens**: a text prompt, a class label, an image, a robot's observation.

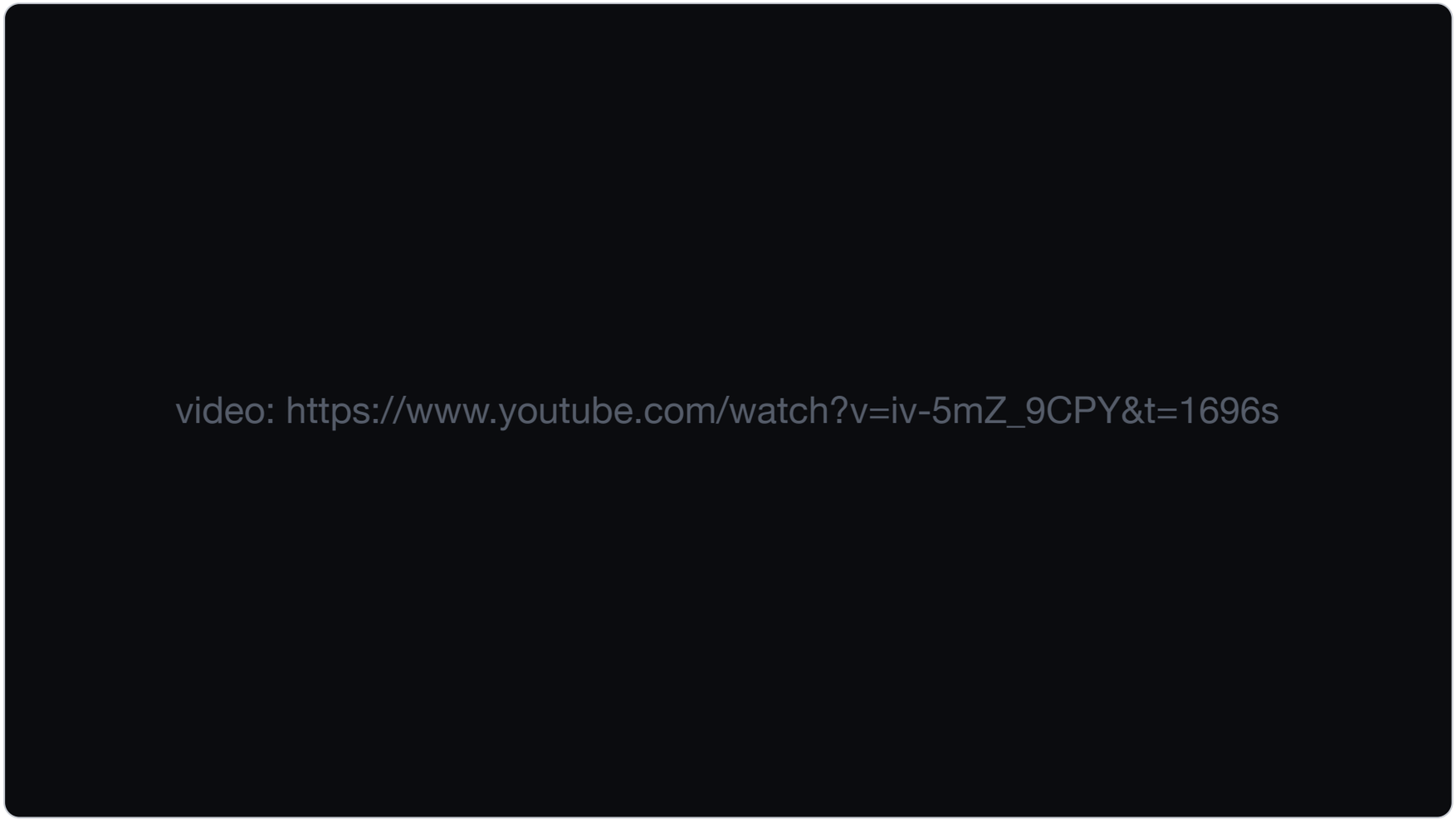
Conditioning on a prompt

- $v_\theta(t, x, y)$: the condition y is **one more input** — a class label, a text embedding, an image, a robot observation
- training pairs (x_1, y) come from the data set
- the loss is unchanged
- inference: fix y and run the same numerical integration loop

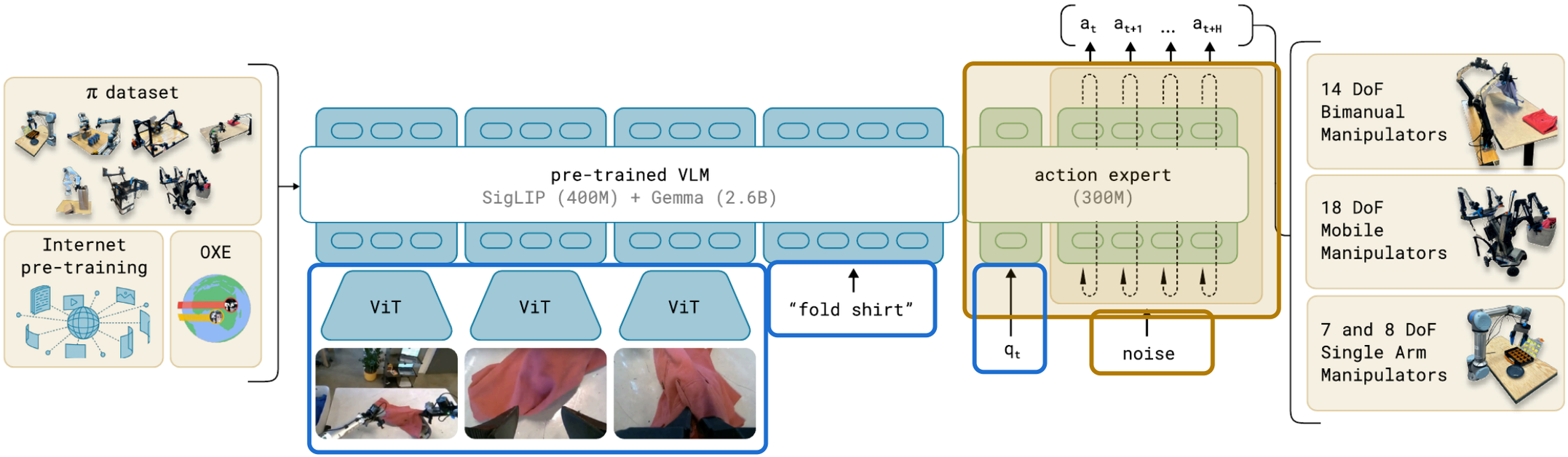


Video: how do AI images and videos work?

Welch Labs, guest video on 3Blue1Brown, from 28:16 · youtube.com/watch?v=iv-5mZ_9CPY



Example application: π0



conditioning o_t: camera images (through ViT), language command, robot state q_t

flow matching expert: denoises the action chunk, starting from noise

Black et al., π0: A Vision-Language-Action Flow Model for General Robot Control, 2024 · figure: Fig. 3 of the paper, annotated

Example application: π0 - idea

Flow Matching

$$\mathbf{A}_t^0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

$$\downarrow$$

$$\cdots$$

$$\downarrow$$

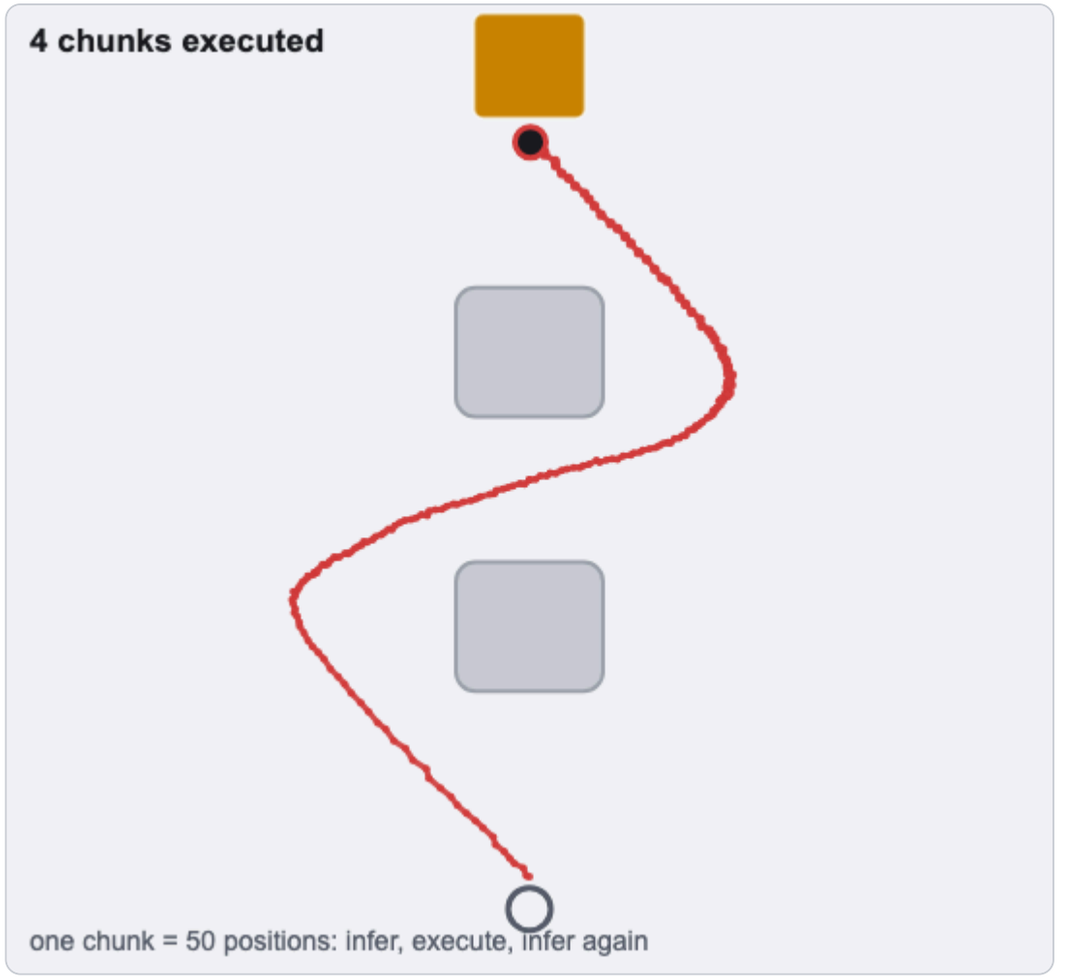
$$\mathbf{A}_t^\tau$$

$$\downarrow$$

$$\cdots$$

$$\downarrow$$

$$\mathbf{A}_t^1 \sim p_1$$



Example application: π0 - how it looks in real life

Flow Matching

$$\mathbf{A}_t^0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

$$\downarrow$$

$$\dots$$

$$\downarrow$$

$$\mathbf{A}_t^\tau$$

$$\downarrow$$

$$\dots$$

$$\downarrow$$

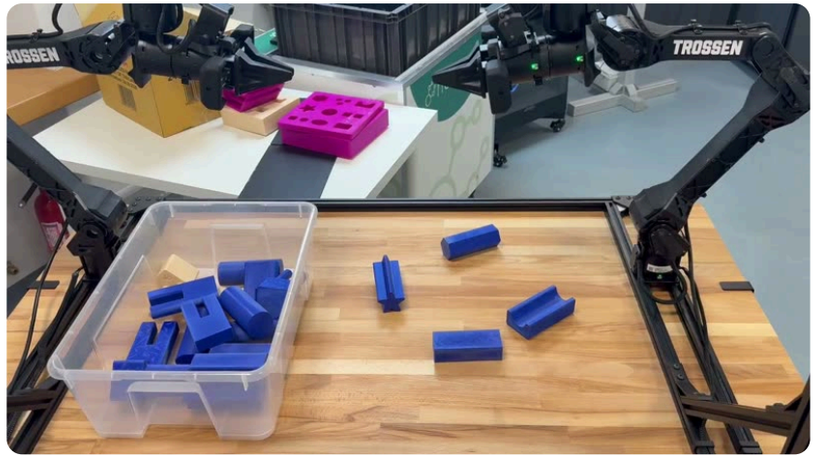
$$\mathbf{A}_t^1 \sim p_1$$



We are doing research in these areas at MIMUW



bimanual box folding



sorting blocks



bin picking

Flow-matching and diffusion policies for bimanual manipulation, trained from demonstrations. If you want to work on this, talk to us.

Summary

- Flow matching **deforms** a known distribution onto the target distribution
- At inference, we draw a sample from the known distribution and numerically integrate the deformation.
- A deformation in time is a **flow**
- Learning the velocity avoids introducing explicit constraints for the network
- Through conditioning we try to learn families of probability distributions at once.

Further reading

- Lipman, Chen, Ben-Hamu, Nickel, Le — Flow Matching for Generative Modeling (ICLR 2023): the paper, with the theorems for Gaussian paths
- Liu, Gong, Liu — Flow Straight and Fast (rectified flow) and Albergo, Vanden-Eijnden — stochastic interpolants (both 2022): closely related formulations
- Holderrieth, Erives — MIT 6.S184: Flow Matching and Diffusion Models: lecture notes and labs
- Fjelde, Mathieu, Dutordoir — An introduction to Flow Matching (Cambridge MLG blog): interactive figures
- Lipman et al. — Flow Matching Guide and Code (2024) and TorchCFM: reference implementations
- Black et al. — π0: A Vision-Language-Action Flow Model for General Robot Control (2024)

Feedback



Please spend a minute: your feedback helps me improve the next lecture.

`tinyurl.com/mimuw-dnn-feedback`

Anonymous, no login, three questions.